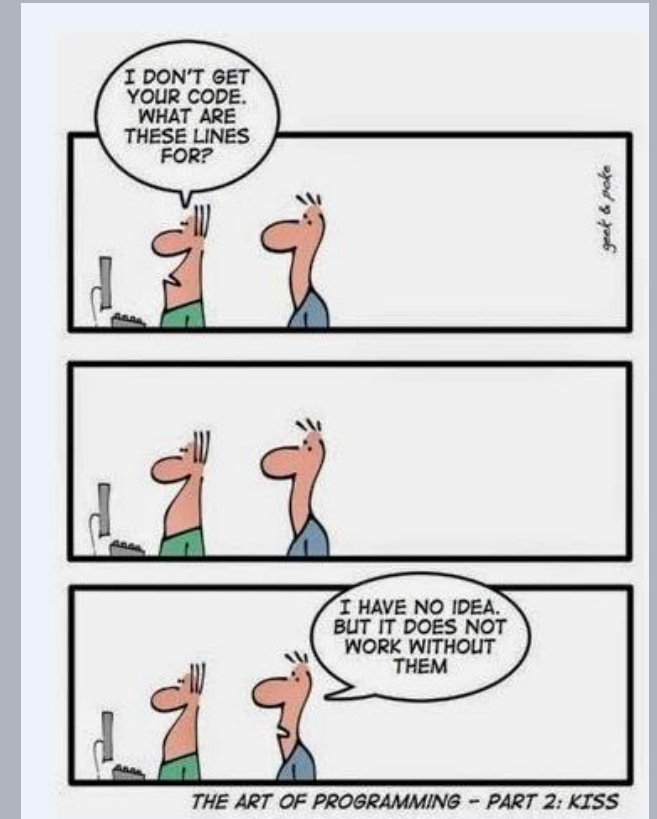


Advanced Software Design Techniques

MISRA and Static Code Analysis

Prof. Dr.-Ing. Peter Fromm



Content

- The problem of code rot
- Why Safer Programming?
- Limitations in C/C++
- Static Code Analysis
- MISRA Standard

Engineering Cycle

Step 1

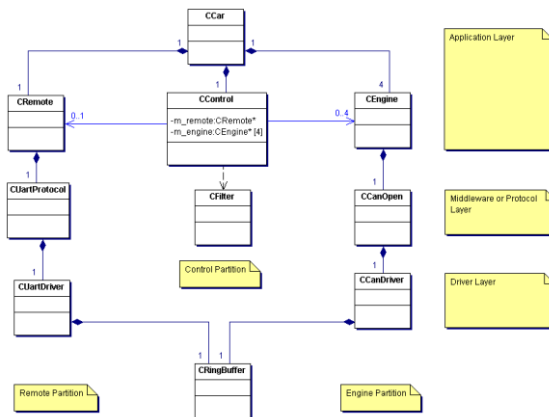
We created a good architecture

Step 2

We started to code

Step 3

We finished to code



Proudly presenting the result...

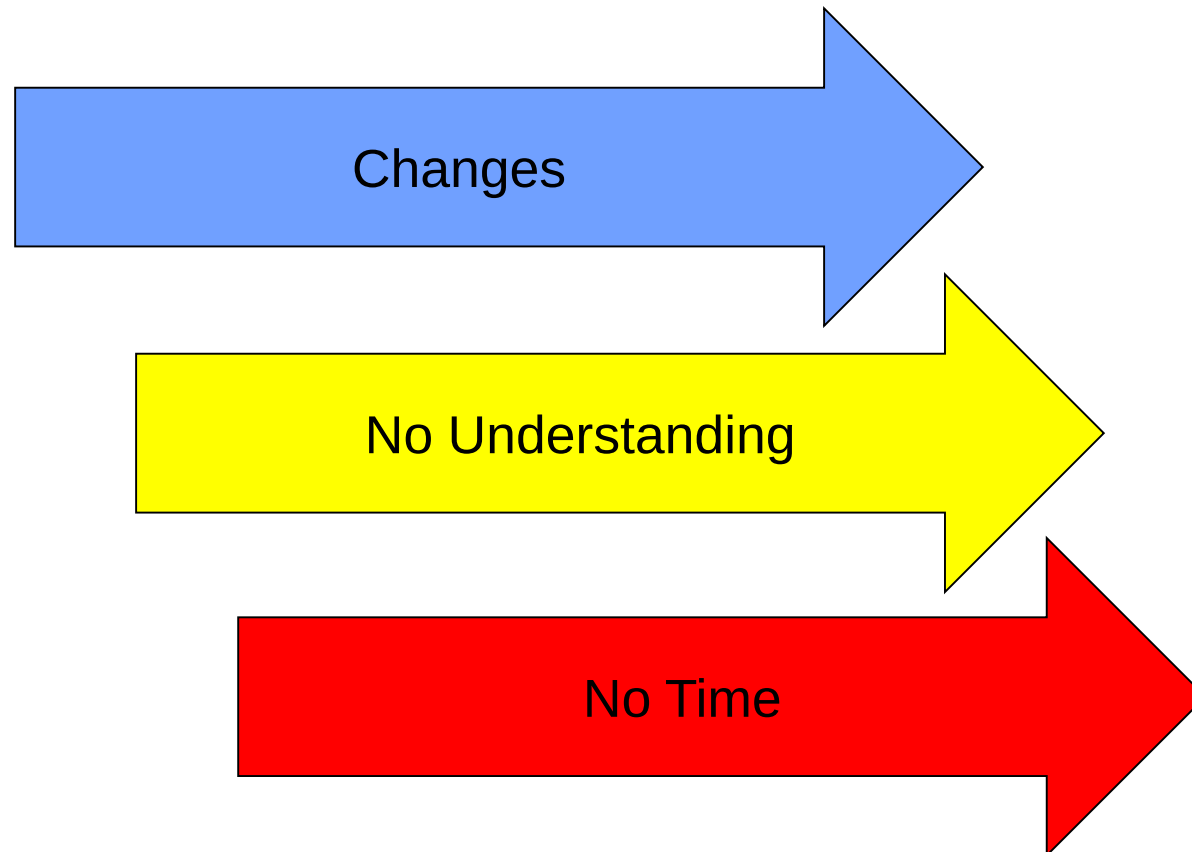
Review the
code



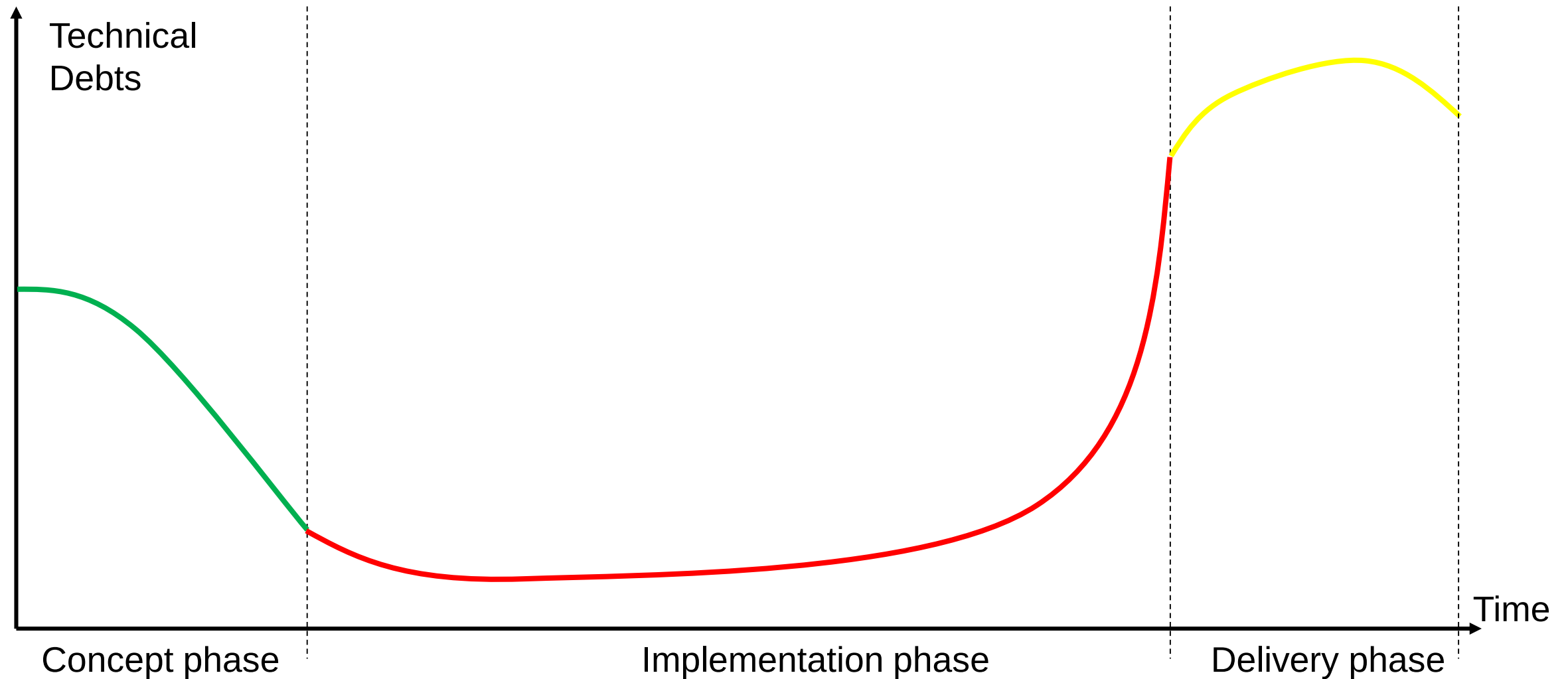
Test the
code

Validation of Tests and Reviews

- Systematic tests and reviews are recommended in all standards
- But still most projects suffer from major code rot at the end of the implementation phase!



Technical debts / code rot / software erosion ...



Reasons for code rot

- Unclear specifications
- Unclear architecture
- Too many changes
- Hacks instead of proper fixes
- Ignored need for refactoring
- Lack of time
- Lack of knowledge
- Different coding styles
- ...

Are we truly navigating?
In the sense – we know, where the
architecture is heading?



Active Rot

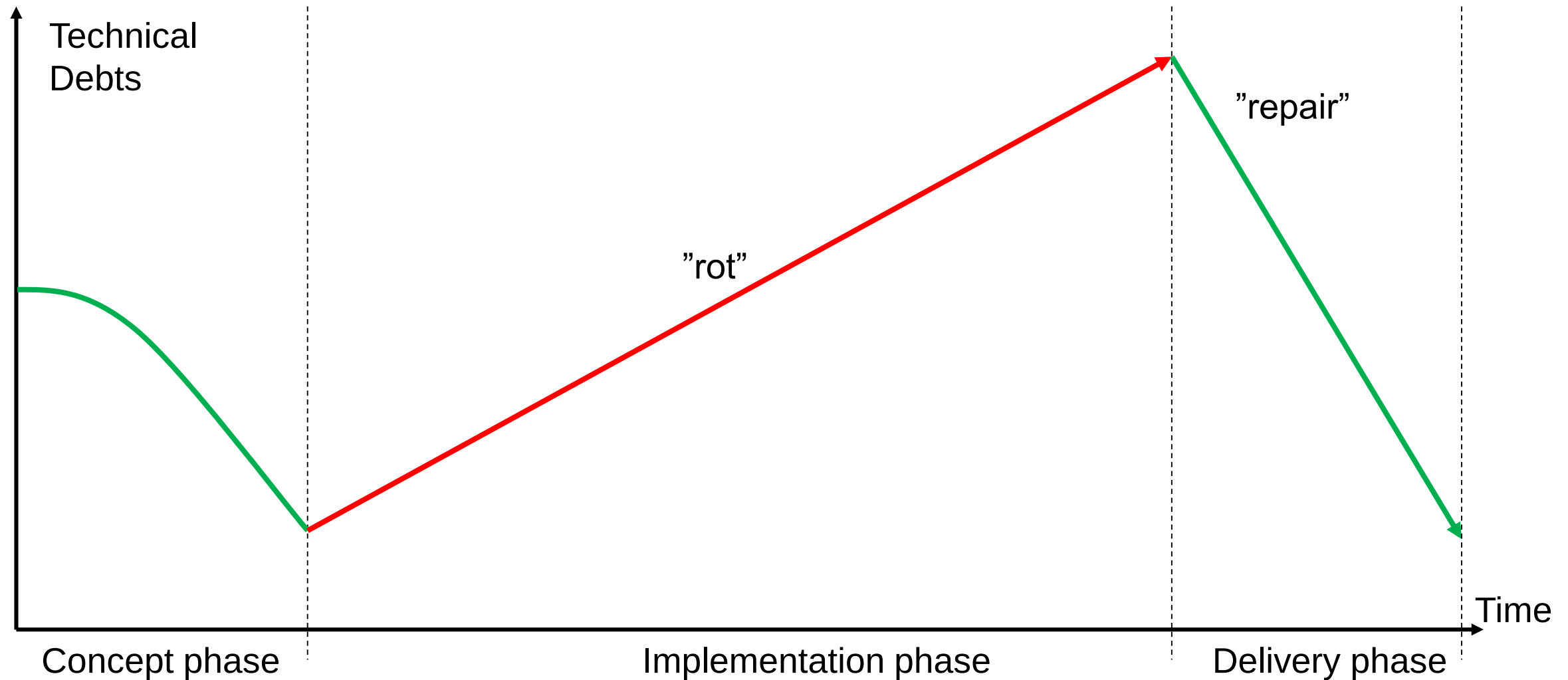
Software that is being continuously modified may lose its integrity over time if proper mitigating processes are not consistently applied. However, much software requires continuous changes to meet new requirements and correct bugs, and re-engineering software each time a change is made is rarely practical. This creates what is essentially an evolution process for the program, causing it to depart from the original engineered design. As a consequence of this and a changing environment, assumptions made by the original designers may be invalidated, introducing bugs.

In practice, adding new features may be prioritized over updating documentation; without documentation, however, it is possible for specific knowledge pertaining to parts of the program to be lost. To some extent, this can be mitigated by following best current practices for coding conventions.

Active software rot slows once an application is near the end of its commercial life and further development ceases. Users often learn to work around any remaining software bugs, and the behaviour of the software becomes consistent as nothing is changing.

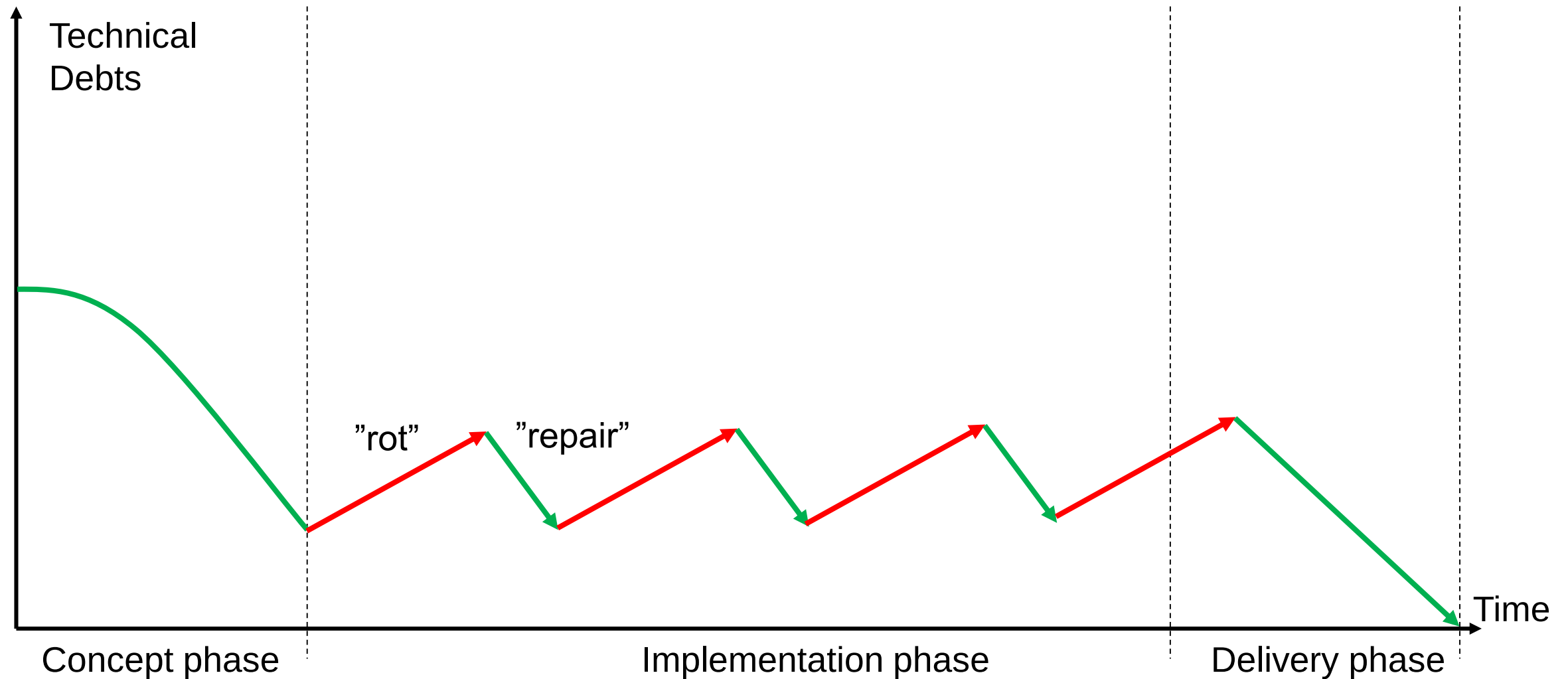
(Wikipedia)

Technical debts – this will not work!



Technical debts

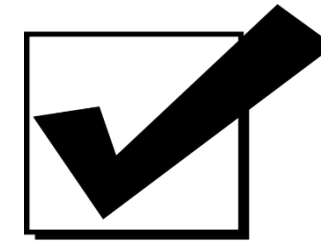
The fast maintenance loop - this might work!



The maintenance loop

- Tests and (to a lesser extend) reviews require very much effort.
- Therefore they are often given a too low priority during the implementation phase!

Reduce debts at the project start → architecture!



Let's write "safer C"

Static Code Analysis

Analyse code rot permanently (and automatically)

Metrics (next lecture)

What do safety standards recommend?

Just do it

Table 1 — Topics to be covered by modelling and coding guidelines

Topics		ASIL			
		A	B	C	D
1a	Enforcement of low complexity ^a	++	++	++	++
1b	Use of language subsets ^b	++	++	++	++
1c	Enforcement of strong typing ^c	++	++	++	++
1d	Use of defensive implementation techniques	o	+	++	++
1e	Use of established design principles	+	+	+	++
1f	Use of unambiguous graphical representation	+	++	++	++
1g	Use of style guides	+	++	++	++
1h	Use of naming conventions	++	++	++	++
^a An appropriate compromise of this topic with other methods in this part of ISO 26262 may be required.					
^b The objectives of method 1b are					
— Exclusion of ambiguously defined language constructs which may be interpreted differently by different modellers, programmers, code generators or compilers.					
— Exclusion of language constructs which from experience easily lead to mistakes, for example assignments in conditions or identical naming of local and global variables.					
— Exclusion of language constructs which could result in unhandled run-time errors.					
^c The objective of method 1c is to impose principles of strong typing where these are not inherent in the language.					

Is C / C++ the best choice?

No programming language can guarantee that the final executable code will behave exactly as the programmer intend.



“The use of C in safety- related or high-integrity systems is not recommended without severe and automatically enforceable constraints. However, if these are present using the formidable tool support for C, the best available evidence suggests that it is then possible to write software of at least as high intrinsic quality and consistency as with other commonly used languages.”

Les Hatton,
“Safer C, Developing Software for High-integrity and Safety-critical Systems”

Code Evolution

C Training 2002:

Why Safer Programming?

Evolution of the amount of software in cars:

1996:	20.000 lines of C code (80.000 lines of assembly language)
1999:	200.000 lines of C code
2002:	estimated: >1.000.000 lines of C code

Some current cluster projects:

– PQ 35	86074*	308787	(B-Sample)
– Lancia LL/ML	15786	77921	(B-Sample)

GM 1999: Recall of 3.5 million cars because of software defect
Old SAT: 10-25 Mio Euro pa for recalls



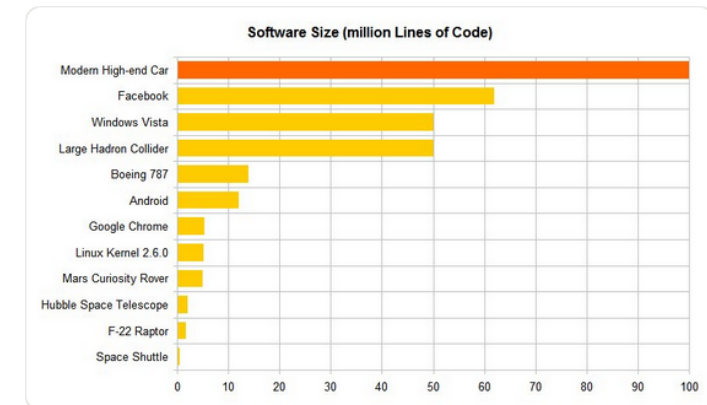
*preprocessed/unpreprocessed LOC

C Programming Guidelines - Slide 5

SV I IP DT DS QE / Dr. P. Fromm

 **Rich Rogers**
@RichRogersIoT [Folgen](#)

"There are 100M lines of code in the average modern car. This should terrify any experienced programmer." - @JosephEames



15:00 - 29. Jan. 2018

1.556 Retweets 1.975 „Gefällt mir“-Angaben



92 1,6 Tsd. 2,0 Tsd.

 **Ryan Schmidt** @rms80 · 30. Jan.
Antwort an @RichRogersIoT @ftrain @josepheames
Apparent source of this number..."probably"... spectrum.ieee.org/transportation...

"it probably contains close to 100 million lines of software code," says Manfred Broy, a professor

A first example (in use since 2002)

The code works – but is it safe?

```
unsigned char kennlinie[255];  
  
void generate_kennlinie(unsigned char a, unsigned char endlevel)  
/* Start with 0 until Point a, then increment until endlevel */  
{  
    int i;  
    /* Calculate Section 2 */  
    for (i=a; i <= a + endlevel; i++)  
    {  
        kennlinie[i] = i-a;  
    }  
    /* Calculate Section 3 */  
    for (i = a + endlevel; i < 255; i++)  
        kennlinie[i] = endlevel;  
}  
  
void main()  
{  
    /* Initial Generation of Kennlinie */  
    generate_kennlinie(10, 40);  
}
```

Magic numbers (pointing to 255)

Uninitialized data (pointing to the first for loop)

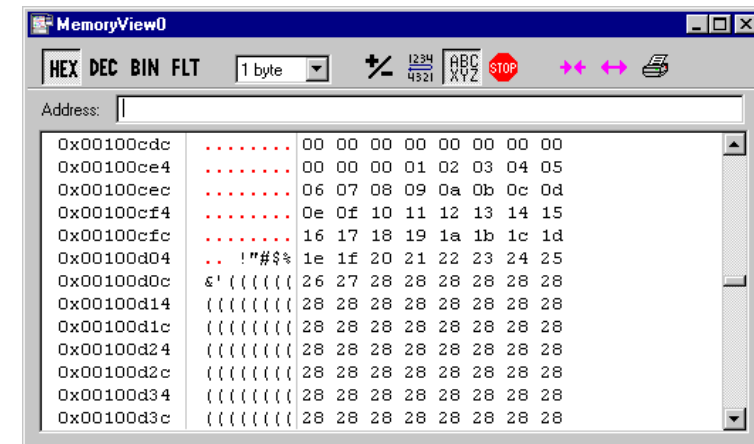
Range overflow (pointing to the second for loop)

Missing brackets (pointing to the second for loop)

C++ comments (pointing to the C++ comments in the code)

endlevel (pointing to the endlevel parameter)

a (pointing to the a parameter)



Limitations in the language definitions of C/C++

Unspecified and undefined behavior

Unspecified behaviour concerns correct program constructs and correct data for which the C standard imposes no requirements. Undefined behaviour concerns illegal program constructs for which the C standard imposes no requirements.

- **values out of range**
- **order of expressions** `a[i] = i++;`
- ...

Implementation-defined behavior

This category concerns well-defined features, which are allowed to vary from compiler implementation to implementation.

- **shift of signed variables**
- **pragmas**
- **pointer casts, bitfields**
- ...

Local-specific behavior

This category concerns features, which are allowed to vary due to international requirements.

- **decimal character**
- **rules for filenames**
- ...

Empirically determined misbehavior

A category of well-defined but frequently abused issues. Also known as the Top 10 ways to get screwed by “C”.

- **assignments if (a=b)**
- **wrong block scope**
- ...

Typedefinitions in C/C++

Compatible type table from Standard (types in each row are compatible)
void
char
signed char
unsigned char
short, signed short, short int, signed short int
unsigned short, unsigned short int
int, signed, signed int, or no type specifiers
unsigned, unsigned int
long, signed long, long int, signed long int
unsigned long, unsigned long int
float
double
long double

ANSI C states:

- No two signed integer types shall have the same rank, even if they have the same representation.
- The rank of a signed integer type shall be greater than the rank of any signed integer type with less precision.
- The rank of long long int shall be greater than the rank of long int, which shall be greater than the rank of int, which shall be greater than the rank of short int, which shall be greater than the rank of signed char.

Use POSIX Types: uint8_t, sint8_t, uint16_t, sint16_t,...

Types, Variables and Pointers

Assignment compatibility - different kinds of conversion

Assignment:

- Applies whenever a value is to be stored in a scalar data object. Explicit type casting is one RECOMMENDED assignment. Often shift operations are used to mask the relevant part of the register. With float/int assignments, data will be lost.

Promotion:

- Small integer types are promoted to natural size when transferring to register. Used with unary +, -, ~ operators and with both operators of a shift operation.

Balancing:

- Applies to any infix operator (e.g. '+') with two operands, which are balanced before the operator is applied. Typically on natural size level with integer types. Complex operations required with big, e.g. long long types!

Types, Variables and Pointers

Implicite Casts

- Assume the following code:

```
U8Variable = FunctionReturningU16();
```

What happens here, some thoughts

- FunctionReturningU16 returns a value > 255: Data ist lost
 - FunctionReturningU16 ALWAYS returns a value < 255: Bad design of the function. Should be corrected!
 - FunctionReturningU16 IN THIS CONFIGURATION always returns a value < 255: In this case use an explicit cast and comment and secure it with compile switches (#if conf != xyz #error...)!
-
- Unfortunately, many projects have an extreme high number of not documented implicite casts!

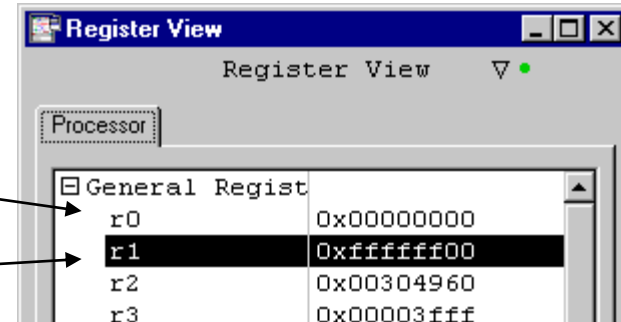
Assignment, Promotion, Balancing

```
unsigned char u8_v1;  
unsigned char u8_v2;  
u8_v1 = ~0xff;  
u8_v2 = 0xff;
```

assignment, promotion

```
if (u8_v1 == ~0xff)  
{  
    // Is this part executed?  
}
```

what about
if (u8_v1 == ~u8_v2)...



```
#include <stdio.h>  
  
void main()  
{  
1   unsigned char    u_ch_v1;  
2   • 0x100098 main:    e92d4300    STMDB    SP!, {R8,R9,LR}  
3   u_ch_v1 = ~0xff;  
4   • 0x10009c main+0x4:  e3a00000    MOV     R0, 0  
5  
6   → if (u_ch_v1 == ~0xff)  
7   • 0x1000a0 main+0x8:  e3e010ff    MVN     R1, 0xff  
8   • 0x1000a4 main+0xc:  e1500001    CMP     R0, R1  
9   • 0x1000a8 main+0x10: 1a000001    BNE     0x4(main+0x1c (0x1000b4))  
10  {  
11      printf("Just did the trick\n");    /* NEVER EXECUTED */  
12  • 0x1000ac main+0x14:  e59f0008    LDR     R0,[PC,8] (0x1000bc)  
13  • 0x1000b0 main+0x18:  eb00000a    BL      0x28(printf (0x1000e0))  
14  }  
15 }
```

Structures, Alignments, Paddings, Bitorder

- The order of members in a class or struct may not be changed!
- But the byte and bitorder as well as alignment and padding is depending on the machine and up to the compiler!

```
struct outer
{
    union inner_1
    {
        unsigned short us1;
        struct {
            unsigned char uc2 : 4;
            unsigned char uc3 : 3;
        } bit;
    }
    struct inner_2
    {
        char a;
        int b;
    }
}
```

Alignment

HighByte,
LowByte?

Padding

How about
sizeof?

Types, Variables and Pointers

Pointers - some considerations

Reasons for using pointers

- Dynamic function calls (e.g. the function to be called is defined during runtime)
- Variable call by reference
- Improved performance

but...

- Pointers are the high potential candidate for implementation bugs
- Pointers decrease maintainability (because they are hard to read)
- Pointer bugs corrupt data
- Pointers bugs are statistical runtime bugs - very hard to find
- Pointer bugs behave different on emulation and flash systems
- ...

Types, Variables and Pointers

Pointers - invalid addresses

Invalid addresses will corrupt the data when pointing to RAM or not performing anything, leading to undefined system behaviour when pointing to ROM

Typical reasons:

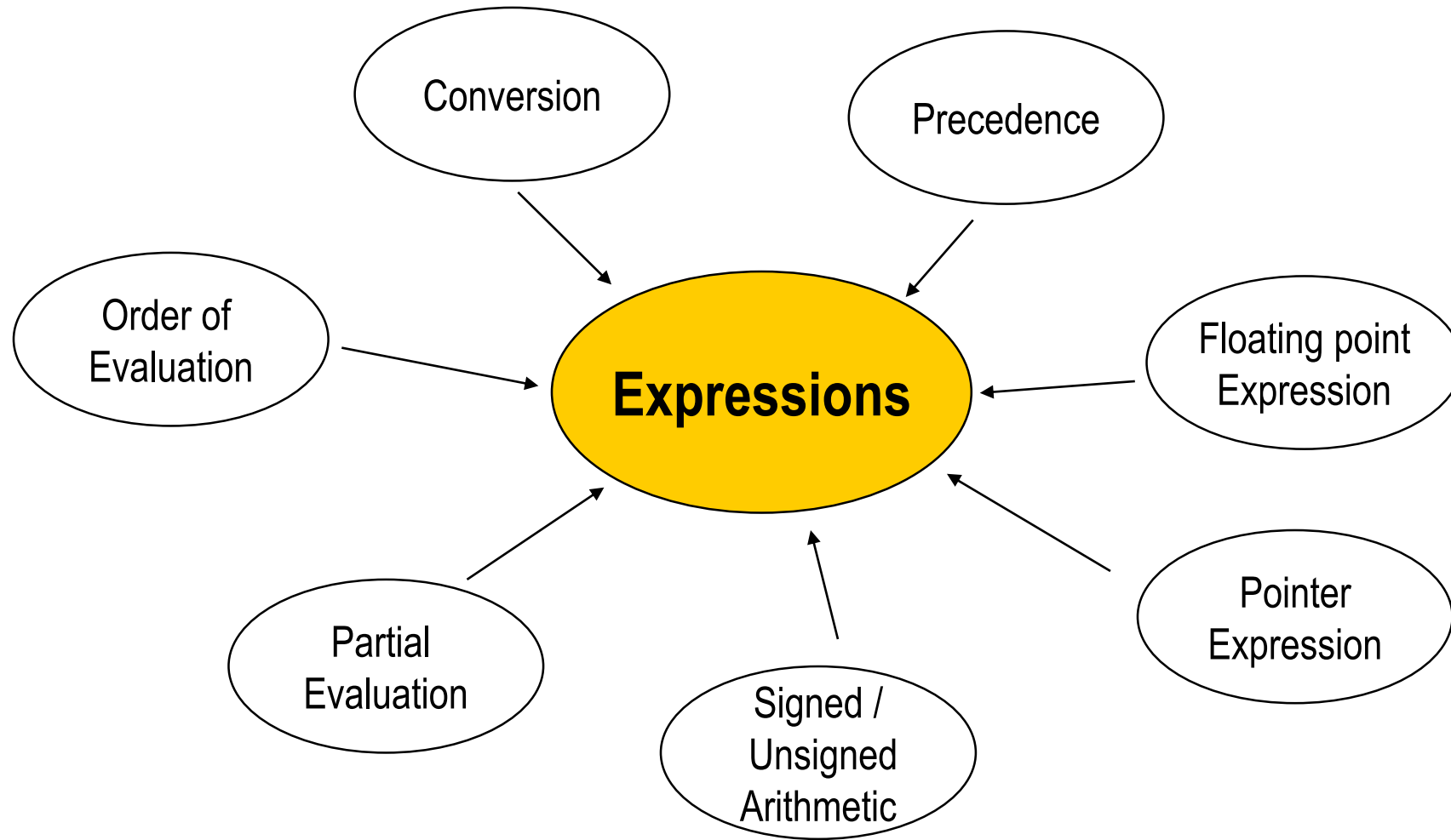
- Pointer is not initialised (Memory has not been allocated or has been deallocated)
- Operation has been priorly performed on pointer instead of contents - `pointer = func()` instead `pointer*=func()`
- Pointer has been casted
- Array access out of bounds
- Wrong indirection
- Invalid calculations (e.g. different object lifetimes)

Types, Variables and Pointers

Pointers - some golden rules

- Make sure the pointers to be used are valid
if (pointer != NULL) { do something }
- The adress of an object should not be stored in an object with larger scope
globalpointer = localpointer + 3; // Fatal outside scope of localpointer
- Avoid pointer indirection deeper than two levels
int **b = *a; // Very hard to map to a human mind
- Avoid casting between different pointer types
- Avoid array accesses out of bounds
read return rubbish data, write corrupts other data

Expressions Overview



Expressions

Operator Precedence Levels

Priority	Operators	Associativity
0	() [] -> .	Left to Right
1	! ~ ++ -- + - * & sizeof (type)	Right to Left
2	* / %	Left to Right
3	+ -	Left to Right
4	<< >>	Left to Right
5	< <= > >=	Left to Right
6	== !=	Left to Right
7	&	Left to Right
8	^	Left to Right
9		Left to Right
10	&&	Left to Right
11		Left to Right
12	?:	Right to Left
13	= += -= *= /= %= &= ^= = <<= >>=	Right to Left
14	,	Left to Right

Use brackets!!!!

Control Flow

- Nested IF structures are hard to read - use indentations and { } for all statements and always implement the easiest logic
- Conditions are forgotten - always provide catch all / default clauses
- Forgotten BREAK statements in SWITCH statements
- Expressions in conditions
- Floats in conditions
- Changing control variables inside the body of a FOR statement
- Using GOTO statements
- Using implicate comparison, e.g. if (func()) ...
- Unclear boolean expressions (On, Off) or redefinition of expressions
- Inverted HW-registers, e.g. 0 = ON, 1 = OFF
- ...

Interfaces - parameters and return statements

Consider: INP_wMittelwert(&wErg, &wData, wNew, 10);

- What information does the return value provide?
- What does this mean in terms of responsibility?

```
BYTE INP_wMittelwert(WORD *wMittelwert, WORD *waAD, WORD wAD, BYTE bLaenge)
{
    BYTE cZaehl;

    /* Check, if array is out of bounds */
    if (bLaenge > 9) return (ERROR);

    wMittelwert = 0;
    /* Alle Werte nach rechts schieben 8->9, 7->8, ... 0->1 */
    for(cZaehl = bLaenge; cZaehl > 0; cZaehl--)
    {
        waAD[cZaehl] = waAD[cZaehl-1];
    }
    waAD[0] = wAD;

    for(cZaehl = 0; cZaehl < bLaenge; cZaehl++)
    {
        wMittelwert += waAD[cZaehl];
    }
    wMittelwert /= bLaenge;

    return(OK);
}
```

Another perspective

Quidquid latine dictum sit, altum viditur.
Whatever is said in Latin sounds profound.

To foil the maintenance programmer, you have to understand how he thinks. He has your giant program. He has no time to read it all, much less understand it. He wants to rapidly find the place to make his change, make it and get out and have no unexpected side effects from the change.

He views your code through a tube taken from the centre of a roll of toilet paper. He can only see a tiny piece of your program at a time. You want to make sure he can never get at the big picture from doing that. You want to make it as hard as possible for him to find the code he is looking for. But even more important, you want to make it as awkward as possible for him to safely **ignore** anything.

Programmers are lulled into complacency by conventions. By every once in a while subtly violating convention you force him to read every line of your code with a magnifying glass.

You might get the idea that every language feature makes code unmaintainable -- not so, only if properly misused.



Roedy Green - How to write unmaintainable code

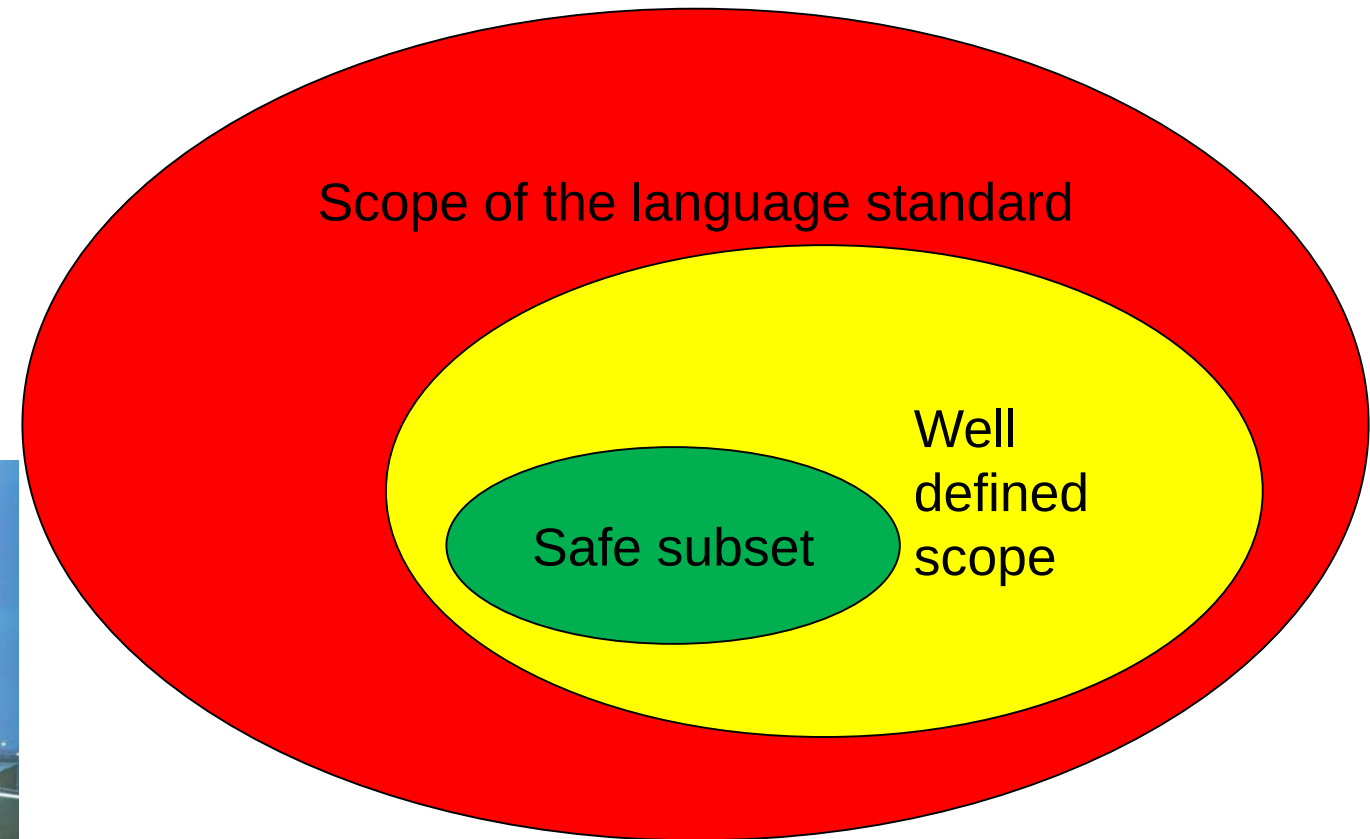
Defensive Programmierung

MISRA C

It is a software development standard for the C programming language. Its aims are to facilitate code portability and reliability in the context of embedded systems, specifically those systems programmed in ANSI C.

Standards and reliability

Even though there is not a *MISRA certification process*, MISRA guidelines are thoroughly followed, especially in automotive industry, as they represent one of the most popular standards for developing secure software.

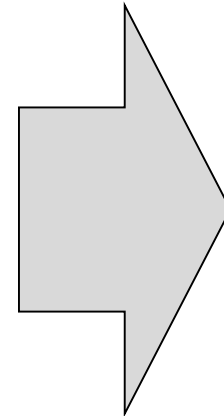
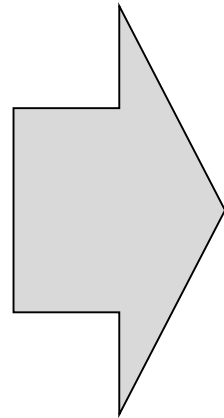


Motor Industry Software Reliability Association MISRA

- Guidelines for tools and documentation
- Data types, operations, casting...
- Program logic
- Structure of functions
- Complex (user defined) data types
- Usage of the preprocessor and standard libraries
- Runtime errors

Static Code Analysis

```
for (int i = 0; i < 10; i++)  
{  
    if (m_pbuffer[i] != 0)  
        m_pBuffer[i]->print();  
}
```



→ **Avoid MAGIC NUMBERS**

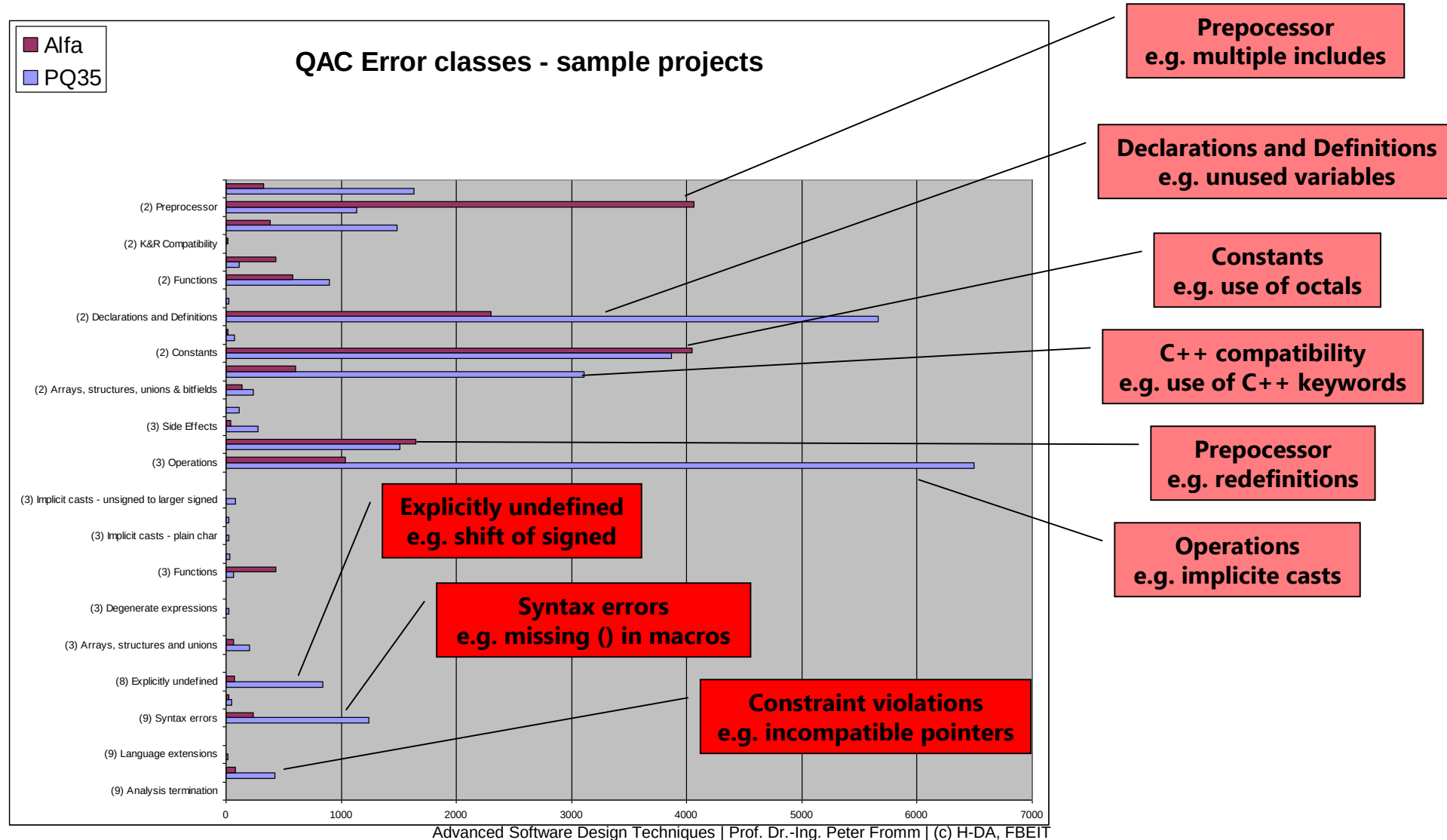
```
for (int i = 0; i < 10; i++)  
{  
    if (m_pbuffer[i] != 0)  
        m_pBuffer[i]->print();  
}
```

Example

The screenshot displays the CodeCheck tool interface for a project named "090_Mira.ubd". The interface is divided into several sections:

- Project Browser:** Located on the left, it shows a tree view of the project files, including folders like "settings" and "myCode", and files like "CAccount.cpp", "CAccount.h", "CATM.cpp", "CATM.h", "CDatabaseReplication.cpp", "CDatabaseReplication.h", "CKeyboard.cpp", "CKeyboard.h", "CMoneySlot.cpp", "CMoneySlot.h", "CScreen.cpp", "CScreen.h", and "main.cpp".
- Files:** A section on the left side of the main window, listing the files in the project.
- Checks:** A section on the left side of the main window, listing the checks performed.
- Result Log:** A section on the left side of the main window, listing the results of the checks.
- Results by File:** A section on the left side of the main window, listing the results of the checks for each file.
- Results by Check:** A section on the left side of the main window, listing the results of the checks for each check.
- Result Treemap:** A section on the left side of the main window, listing the results of the checks for each file.
- Ignore List:** A section on the left side of the main window, listing the results of the checks for each file.
- Result Locator:** A table in the center of the main window, listing the results of the checks. The table has columns for File, Violation, Line, Column, CheckID, Check Name, and Entity. It contains a large number of entries, each representing a violation found in the code.
- Information Browser:** A section on the bottom left, showing the directory structure of the project and the file names.
- Analysis Log:** A section on the bottom right, showing the results of the analysis, including the completion time and the files analyzed.

Some reallife data



MISRA 6.3 Types

Rule 6.3 (advisory): *typedefs* that indicate size and signedness should be used in place of the basic numerical types.

The basic numerical types of *signed* and *unsigned* variants of *char*, *int*, *short*, *long* and *float*, *double* should not be used, but specific-length *typedefs* should be used. Rule 6.3 helps to clarify the size of the storage, but does not guarantee portability because of the asymmetric behaviour of integral promotion. See discussion of integral promotion — section 6.10. It is still important to understand the integer size of the implementation.

Programmers should be aware of the actual implementation of the *typedefs* under these definitions.

For example, the ISO (POSIX) *typedefs* as shown below are recommended and are used for all basic numerical and character types in this document. For a 32-bit integer machine, these are as follows:

```
typedef      char    char_t;
typedef signed  char  int8_t;
typedef signed short int16_t;
typedef signed  int   int32_t;
typedef signed  long  int64_t;
typedef unsigned char  uint8_t;
typedef unsigned short uint16_t;
typedef unsigned int    uint32_t;
typedef unsigned long   uint64_t;
typedef      float     float32_t;
typedef      double    float64_t;
typedef long    double float128_t;
```

typedefs are not considered necessary in the specification of bit-field types.

MISRA 8.7 Scope

Rule 8.7 (required): Objects shall be defined at block scope if they are only accessed from within a single function.

The scope of objects shall be restricted to functions where possible. File scope shall only be used where objects need to have either internal or external linkage. Where objects are declared at file scope Rule 8.10 applies. It is considered good practice to avoid making identifiers global except where necessary. Whether objects are declared at the outermost or innermost block is largely a matter of style. “Accessing” means using the identifier to read from, write to, or take the address of the object.

MISRA Implicite Casts

- **Type widening in integral promotion:** The type in which integral expressions are evaluated depends on the type of the operands after any integral promotion. It is always possible to multiply two 8-bit values and access a 16-bit result if the magnitude requires it. It is sometimes, but not always, possible to multiply two 16-bit values and retrieve a 32-bit result. This is a dangerous inconsistency in the C language and in order to avoid confusion it is safer never to rely on the widening type afforded by integral promotion. Consider the following example:

```
uint16_t u16a = 40000;    /* unsigned short / unsigned int ?    */
uint16_t u16b = 30000;    /* unsigned short / unsigned int ?    */
uint32_t u32x;            /* unsigned int / unsigned long ?     */
u32x = u16a + u16b;       /* u32x = 70000 or 4464 ?             */
```

The expected result is presumably 70000, but the value assigned to `u` will in practice depend on the implemented size of an *int*. If the implemented size of an *int* is 32 bits, the addition will occur in 32-bit signed arithmetic and the correct value will be stored. If the implemented size of an *int* is only 16 bits, the addition will take place in 16-bit unsigned arithmetic, wraparound will occur and will yield the value 4464 (70000 % 65536). Wraparound in unsigned arithmetic is well defined and may even be intended; but there is potential for confusion.

Rule 16.10 (required): **If a function returns error information, then that error information shall be tested.**

A function (whether it is part of the standard library, a third party library or a user defined function) may provide some means of indicating the occurrence of an error. This may be via an error flag, some special return value or some other means. Whenever such a mechanism is provided by a function the calling program shall check for the indication of an error as soon as the function returns.

However, note that the checking of input values to functions is considered a more robust means of error prevention than trying to detect errors after the function has completed (see Rule 20.3). Note also that the use of *errno* (to return error information from functions) is clumsy and should be used with care (see Rule 20.5).

Wrapup - Application of Programming Rules

Avoiding problems

- building up know-how
- considering the rules during implementation
- defining subsets

Controlling the application of the rules

- using compiler warnings
- using dynamic runtime checking
- using MISRA checkers
- using checklists
- using reviews

Improving the rules

- verification of the rules in the project
- communication of problems

